

BRUTE-FORCING A STAY-LOGGED-IN COOKIE



Lab: Brute-forcing a stay-logged-in cookie

This lab allows users to stay logged in even after they close their browser session. The cookie used to provide this functionality is vulnerable to brute-forcing.

To solve the lab, brute-force Carlos's cookie to gain access to his **My account** page.

- Your credentials: wiener:peter
 - Victim's username: carlos
 - Candidate passwords
-

Introduction

First, we need to understand how the application's stay-logged-in cookie works.

When we log in as **wiener** with the **Stay logged in** option enabled, the server creates a cookie called **stay-logged-in**

This cookie is important because the application uses it to recognize us as an authenticated user without requiring us to log in again.

The important part is that the cookie is not random.

After decoding the cookie from Base64, we get:

wiener:51dc30ddc473d43a6011e9ebba6ca770

The first part is the username: **wiener**

The second part looks like an MD5 hash: **51dc30ddc473d43a6011e9ebba6ca770**

We can confirm that this is the MD5 hash of **peter**.

So we discover that the cookie is created like this:

Base64(username + ":" + MD5(password))

For example:

wiener:peter

becomes:

wiener:MD5(peter)

and then the entire value is Base64 encoded.

The problem is that if we know the username and have a list of possible passwords, we can generate the possible cookie values ourselves.

We can then send those cookies to the application and see which one authenticates us as Carlos.

Important Observation

The cookie is effectively acting like an authentication credential.

If we obtain a valid stay-logged-in cookie for another user, we can access that user's account without knowing their password.

There are two important weaknesses in this lab:

1. **Predictable cookie generation**

The cookie is based directly on: **username + ":" + MD5(password)**

This makes it possible to reproduce the cookie.

2. **No effective brute-force protection**

The application allows us to try many different cookie values without blocking or slowing down the attack.

Because of these two weaknesses, we can brute-force Carlos's cookie.

Solution

Step 1: Log in as Wiener

Open the lab and go to **My Account**.



Log in using:

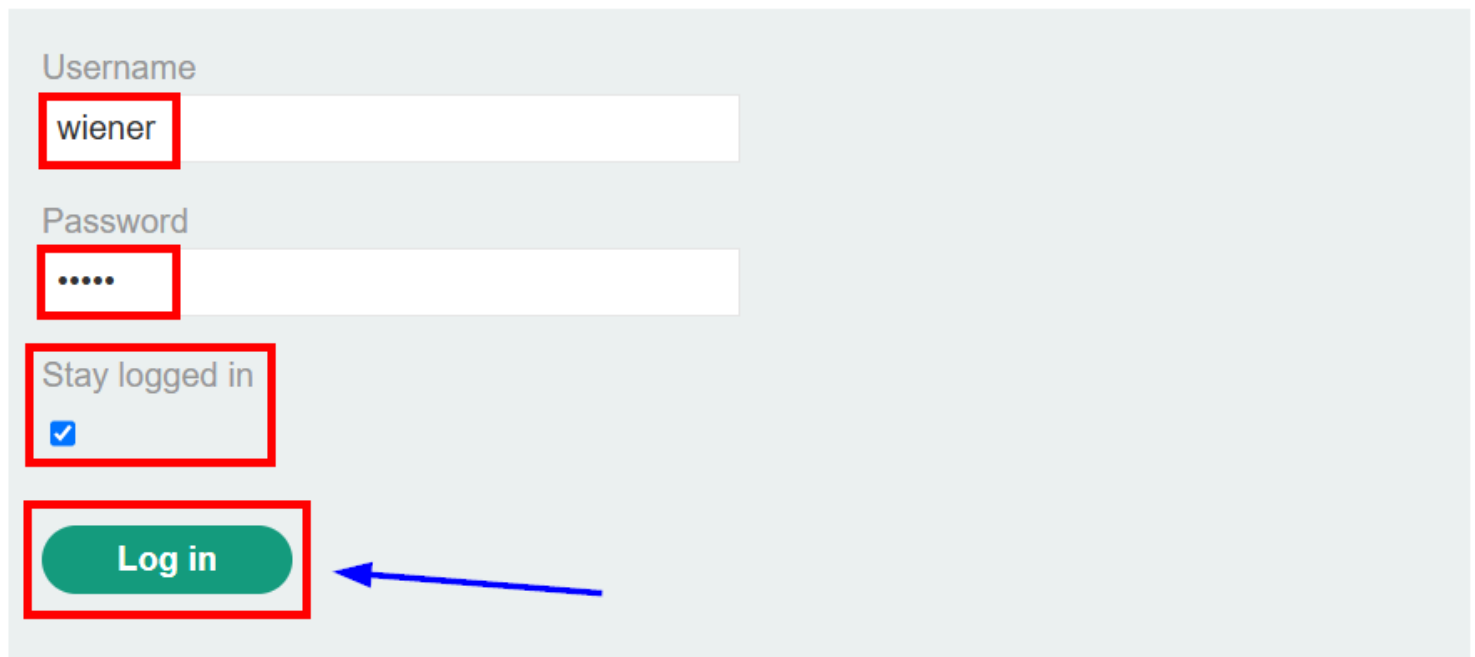
Username: wiener

Password: peter

Before logging in, enable **Stay logged in**

Then click **Log in**.

Login



Username

wiener

Password

.....

Stay logged in

☒

Log in

After successfully logging in, we can see the **Update email** option on the My Account page.

My Account

Your username is: wiener

Email

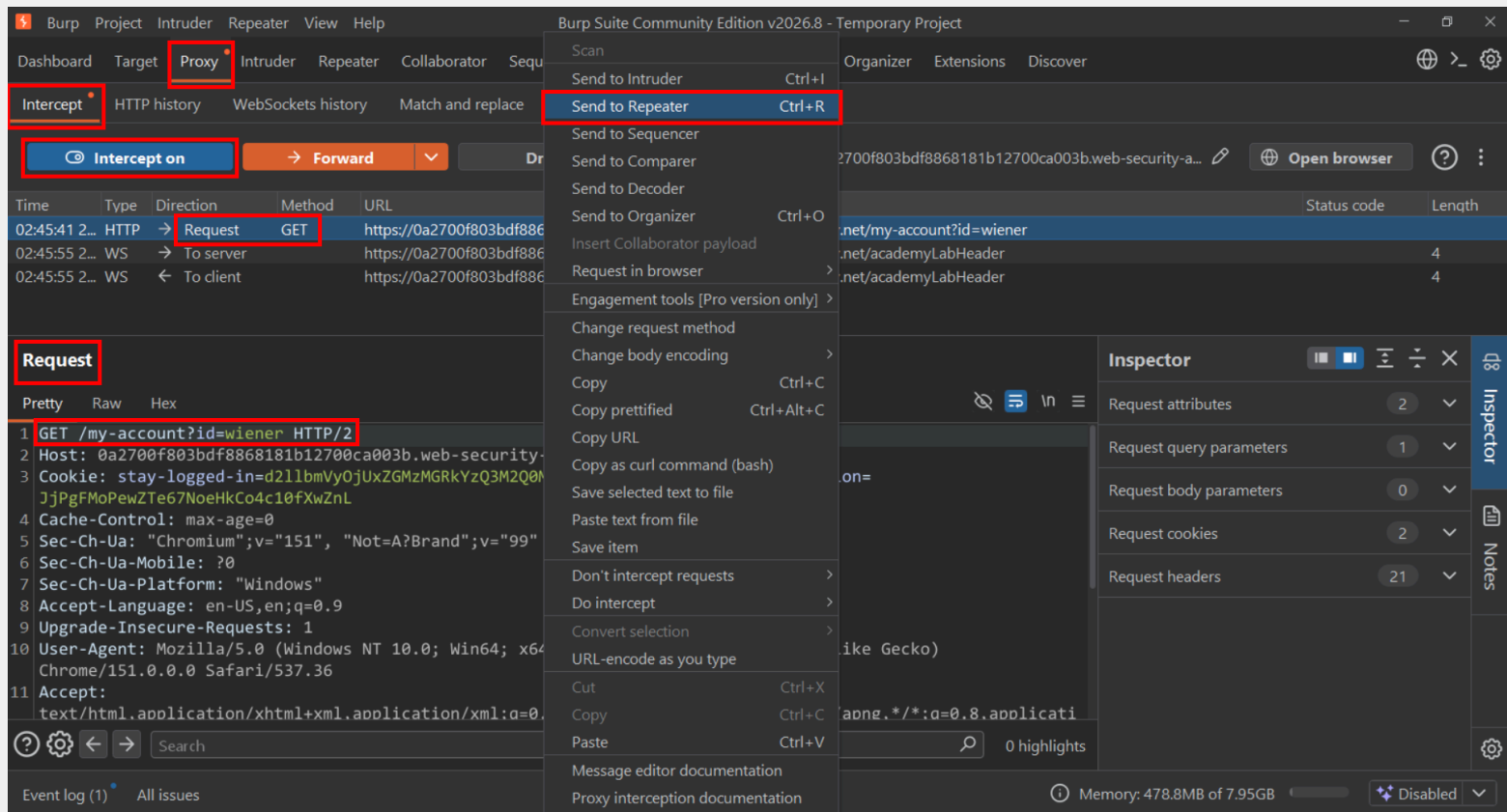
Update email

We will later use the presence of **Update email** to identify when our brute-force attempt is successful.

Update email is not something specific. We can also use something else as an indication of whether the user got logged in or not. Whatever is present on that page after the login will work.

Step 2: Intercept the Request

Open Burp and turn Intercept on and reload the page and a request to **GET /my-account?id=wiener** will be intercepted. Send it to the Repeater.



Step 3: Find the stay-logged-in cookie

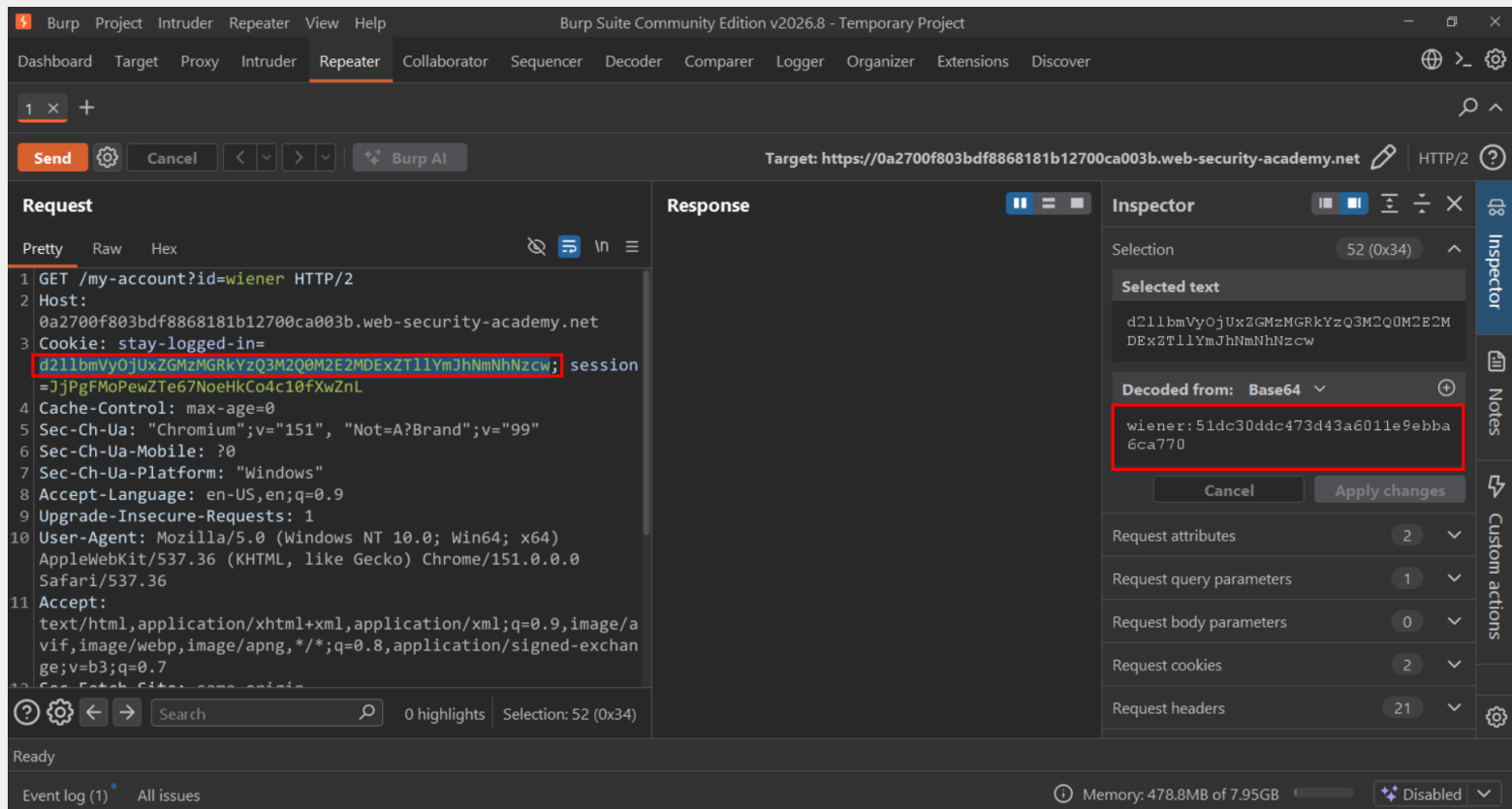
Go to Repeater and look at the request's Cookie header.

We should see something similar to **stay-logged-in=<encoded-value>**

This is the cookie we need to understand.

Step 4: Decode the cookie

Highlight the value of stay-logged-in cookie parameter. You can see on the left side that Burp automatically detects that it is a **Base64** encoded string and decodes it to **wiener:51dc30ddc473d43a6011e9ebba6ca770**



If Burp doesn't automatically detect it, copy the encoded value and send it to Decoder tab and decode it using each of the decoding methods. But since Burp detects it, we don't need to do that and let's go ahead.

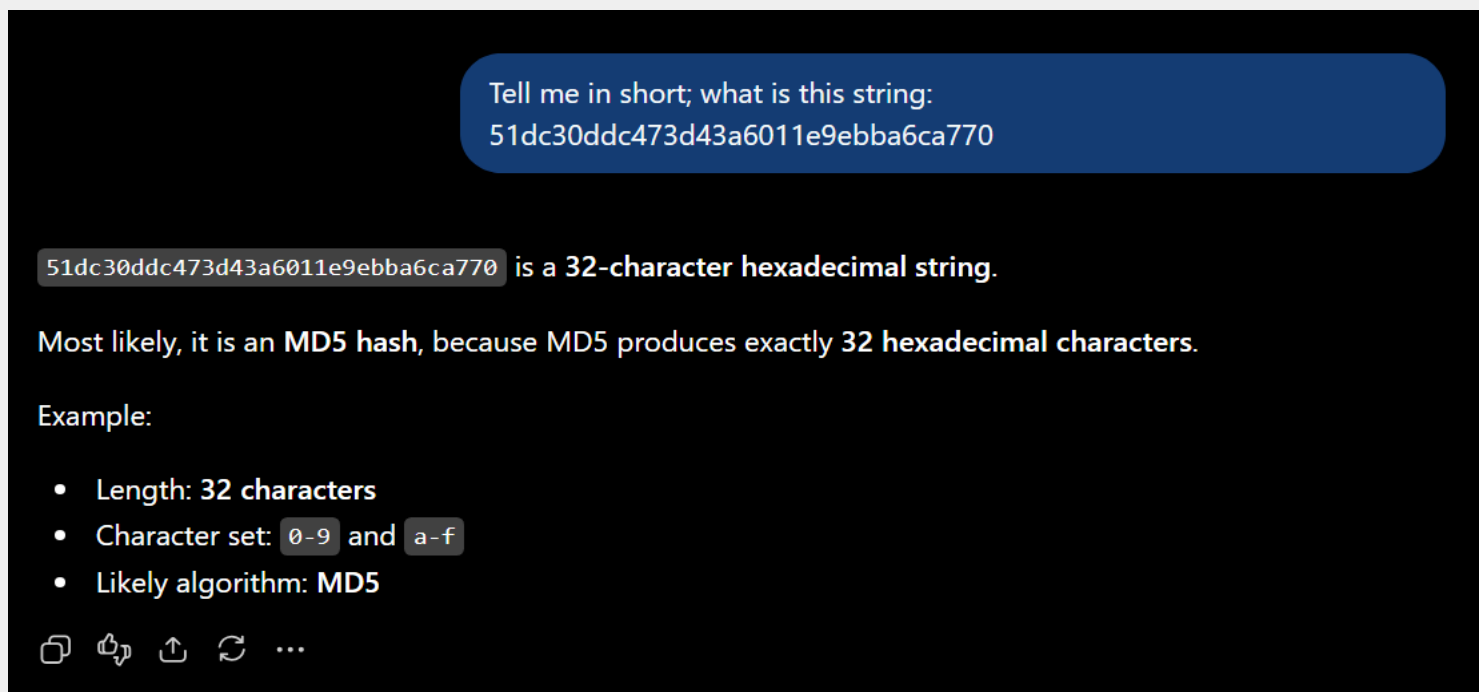
The value has two parts separated by a colon(:):

wiener and **51dc30ddc473d43a6011e9ebba6ca770**

The first part is obviously the username.

The second part is a 32-character hexadecimal string.

So to know what it is, just copy it and ask ChatGPT or any other AI what this is.



This is the ChatGPT response about this string I asked. And it is saying that it is probably an MD5 Hash.

And you can also verify it manually like:

We already know our password is **peter**

We can calculate its MD5 hash.

On Linux, we can use **echo -n "peter" | md5sum**

The result is **51dc30ddc473d43a6011e9ebba6ca770**

This matches the second part of the decoded cookie.

Therefore, we have confirmed the cookie structure:

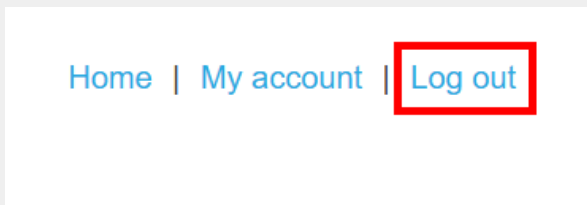
Base64(username + ":" + MD5(password))

So for Wiener **Base64(wiener:MD5(peter))**

Step 5: Prove that we can create the cookie ourselves

Before attacking Carlos, we should test whether we can recreate our own stay-logged-in cookie.

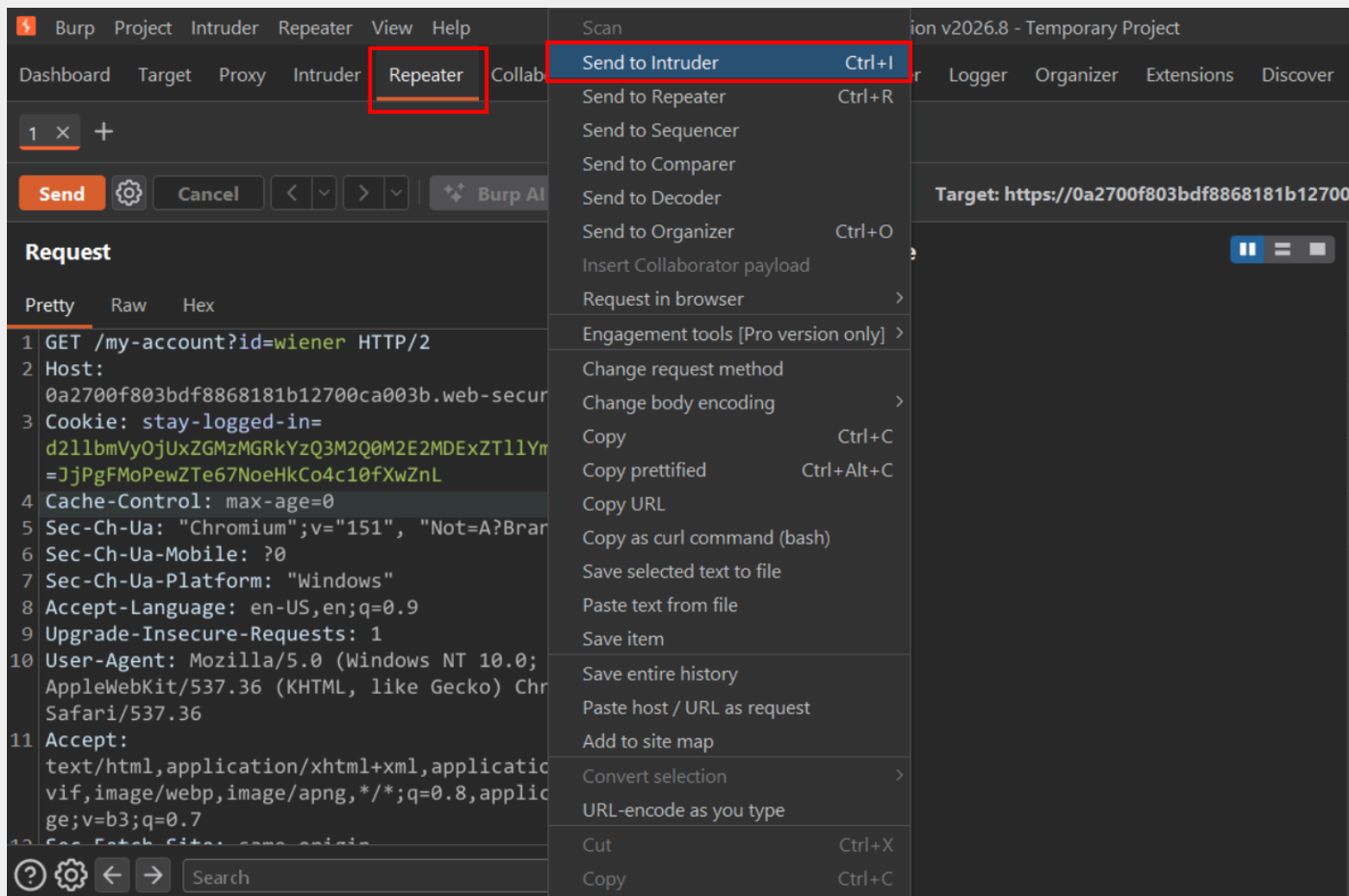
Log out of the application.



Take the request:

GET /my-account?id=wiener

from repeater and send it to **Intruder**



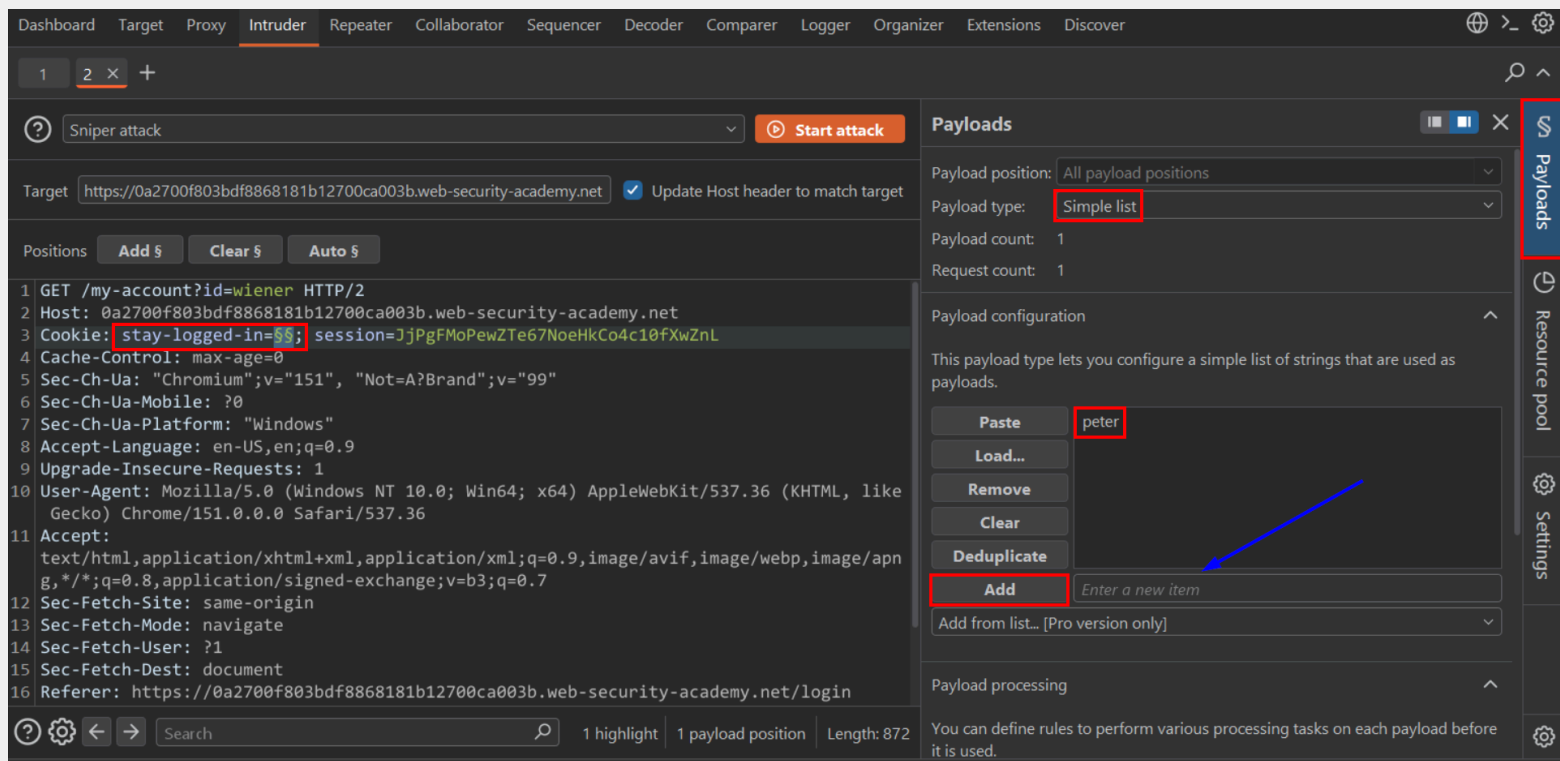
Remove the existing **stay-logged-in** value and mark the value as a payload position.

We will use our password **peter** as the test payload.

Let's go to Payloads.

In the **Payload settings** select **Simple List** and **Add** a password **peter**.

For that, enter any password (here **peter**) in the box at the front of **Add** option and then hit Add.

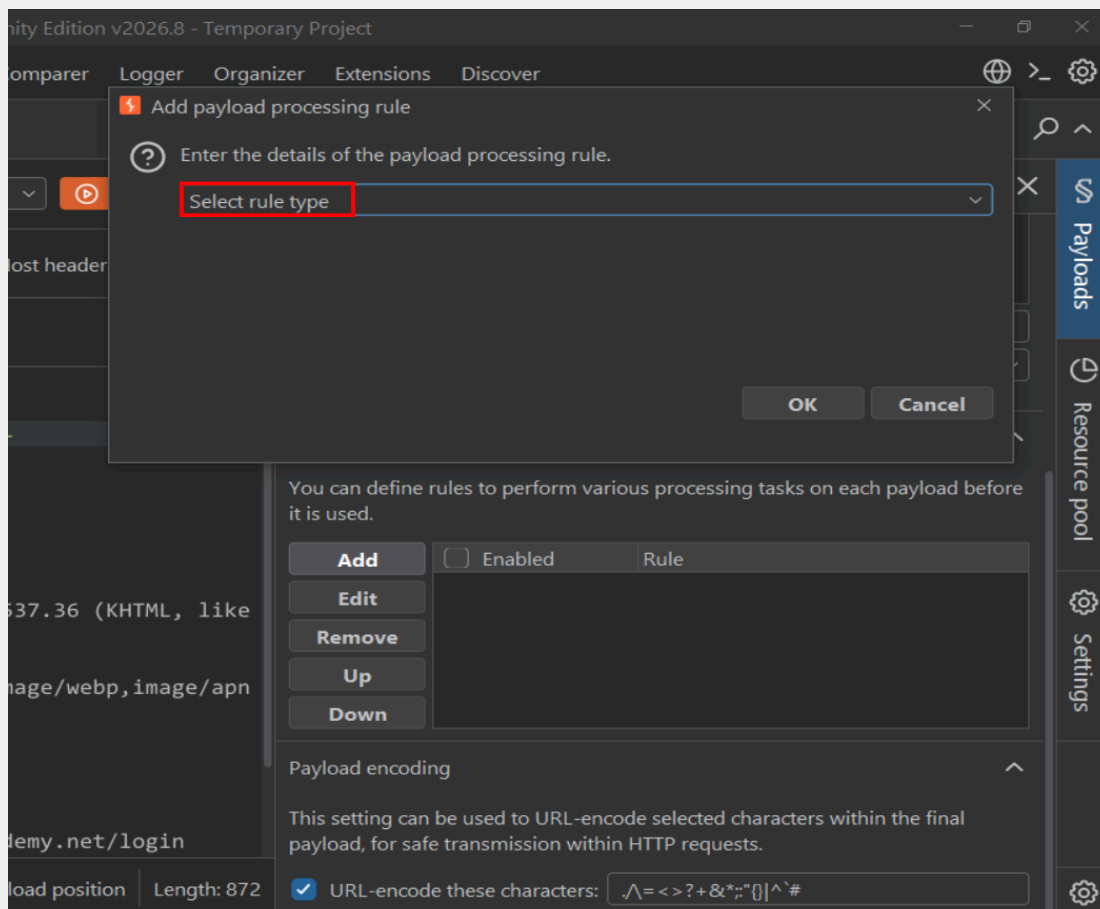


We can use payload processing to modify this value (password)

Under Payload Processing we are going to be adding some rules. Then these rules get applied to this password.

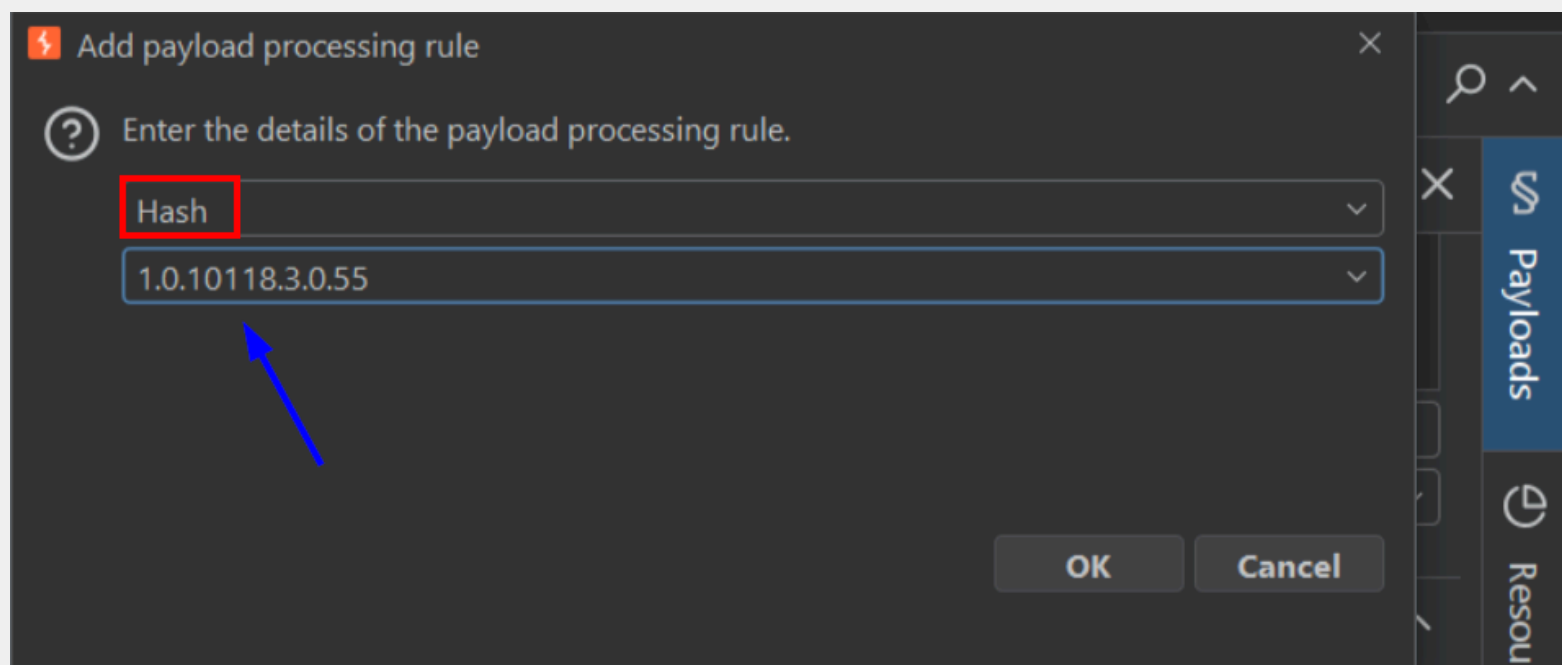
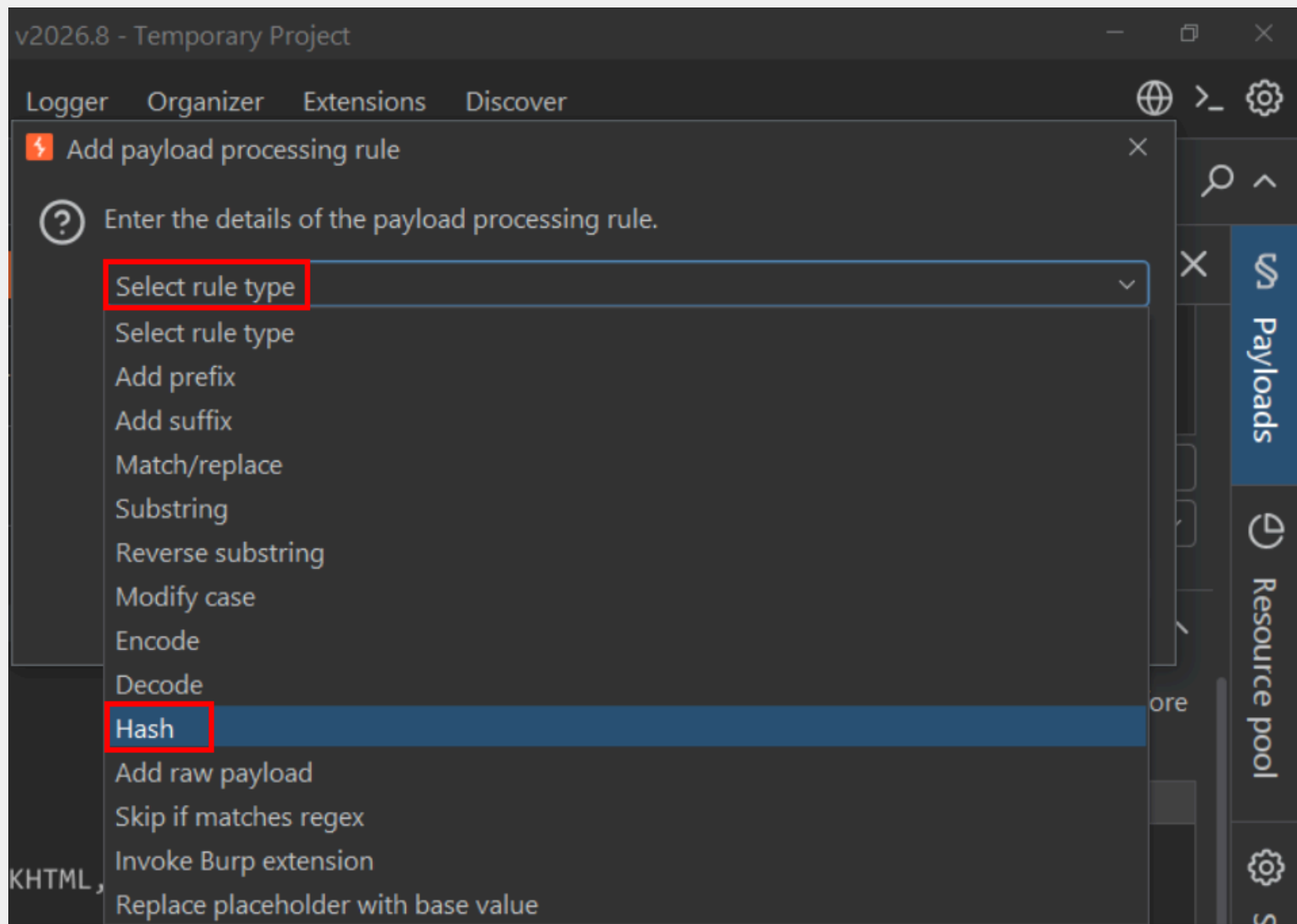
We know that the password is MD5 hashed in the stay-logged-in cookie value.

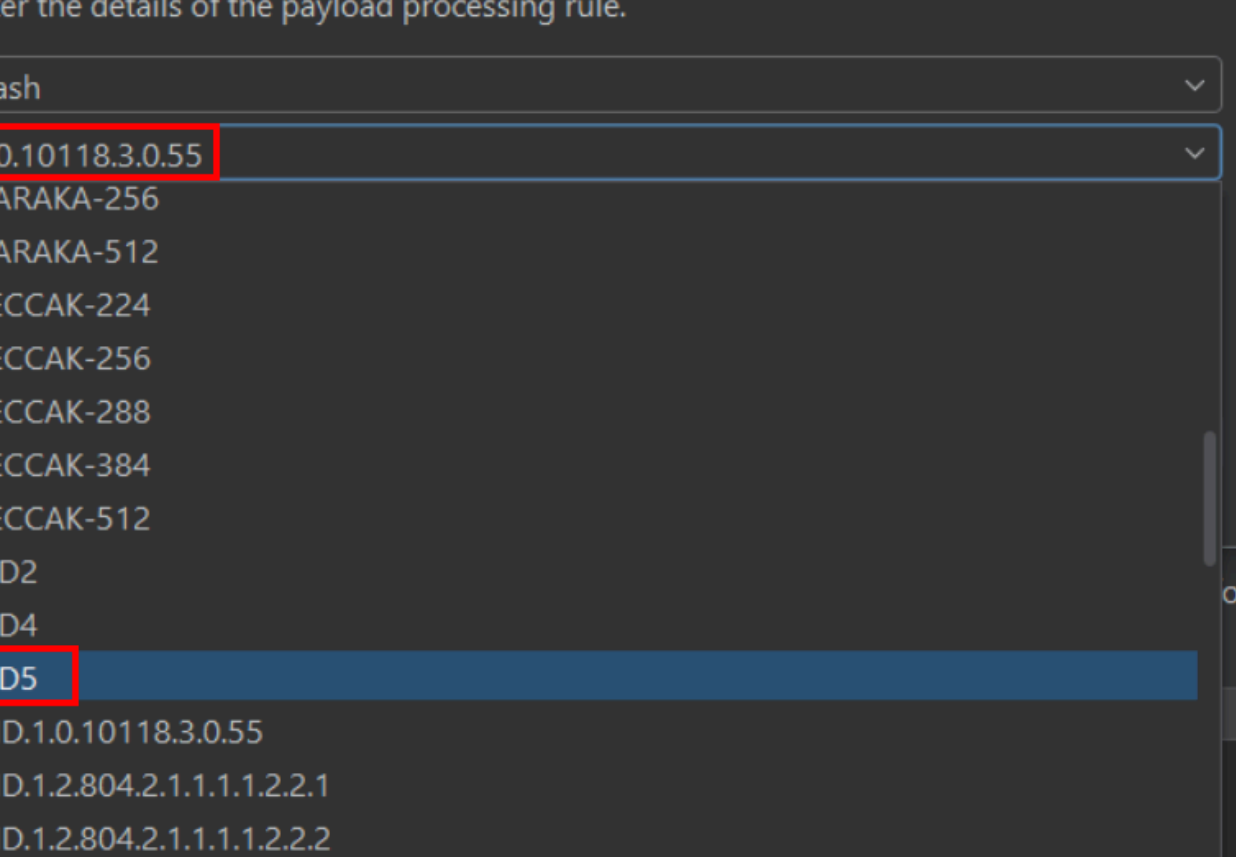
So for that, in the **Payload Processing** click Add and a box (Add payload processing rule) will appear.



These rules will apply to all the entries in the Simple List, but because we have one entry in the list right now that's why it's only gonna apply to that.

Click on Select rule type and Choose Hash.





Add payload processing rule

Enter the details of the payload processing rule.

Hash

1.0.10118.3.0.55

HARAKA-256

HARAKA-512

KECCAK-224

KECCAK-256

KECCAK-288

KECCAK-384

KECCAK-512

MD2

MD4

MD5

OID.1.0.10118.3.0.55

OID.1.2.804.2.1.1.1.2.2.1

OID.1.2.804.2.1.1.1.2.2.2

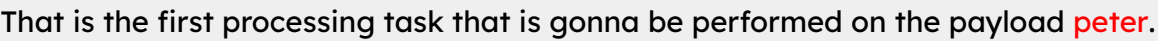
OID.1.2.804.2.1.1.1.2.2.3

OID.2.16.840.1.101.3.4.2.10

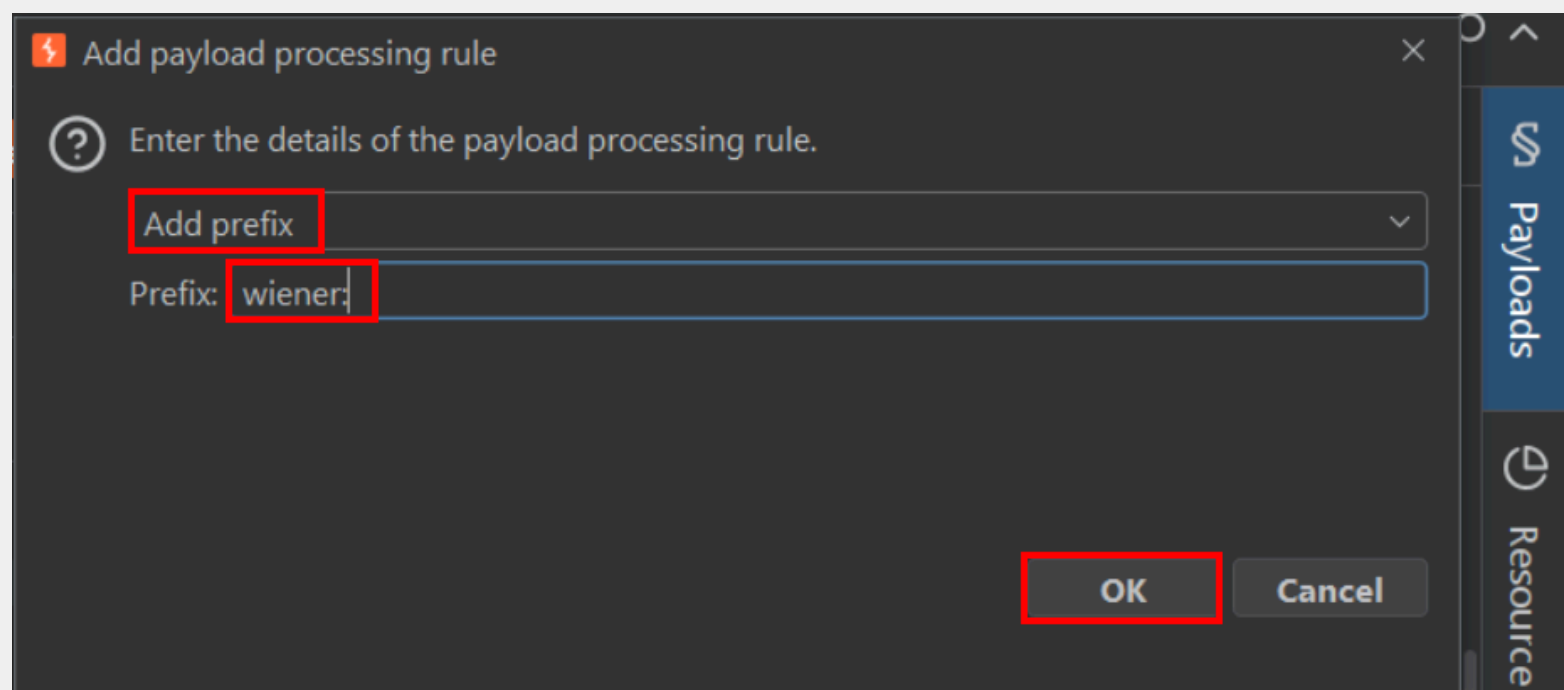
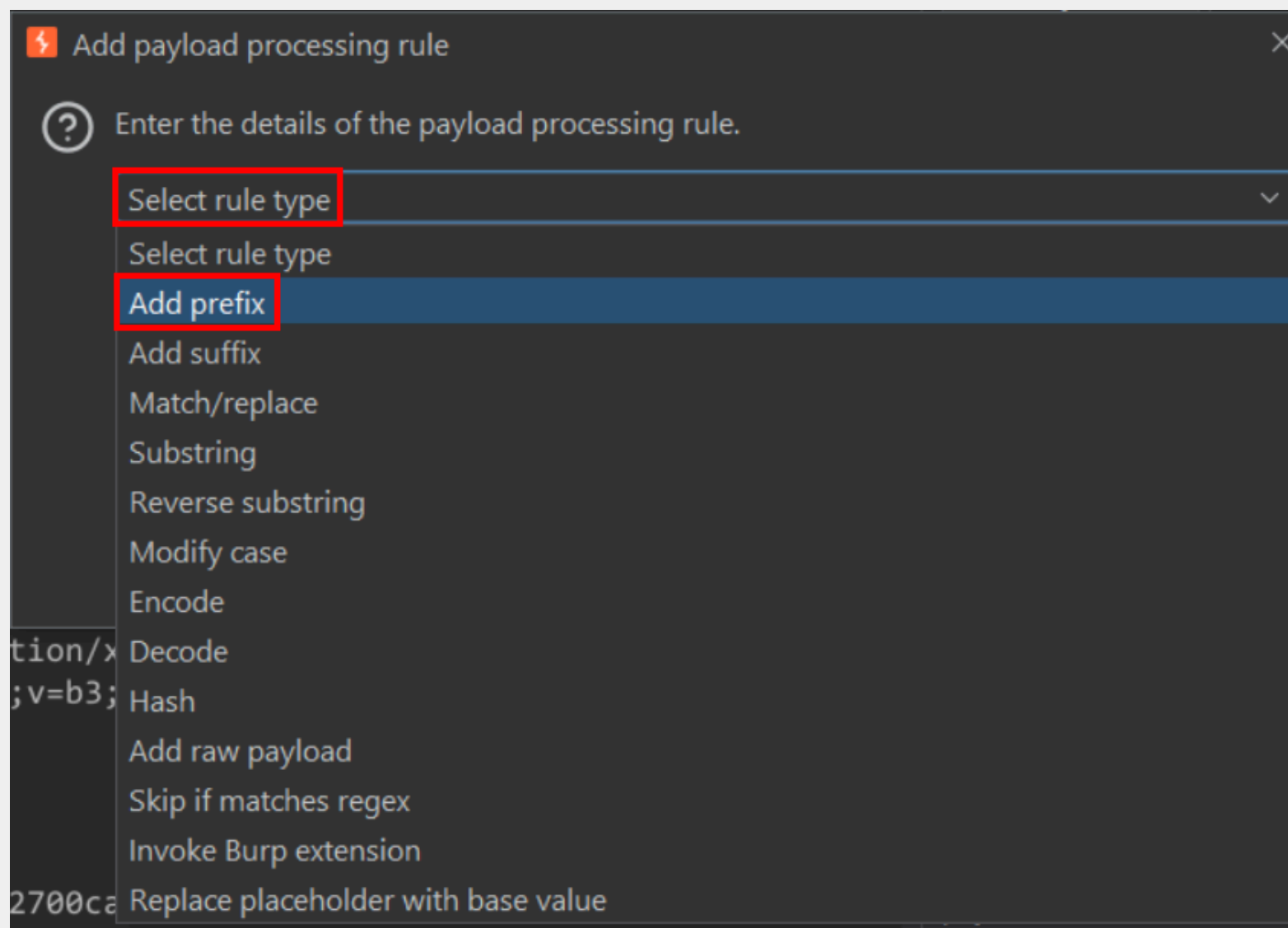
Payloads

Resource pool

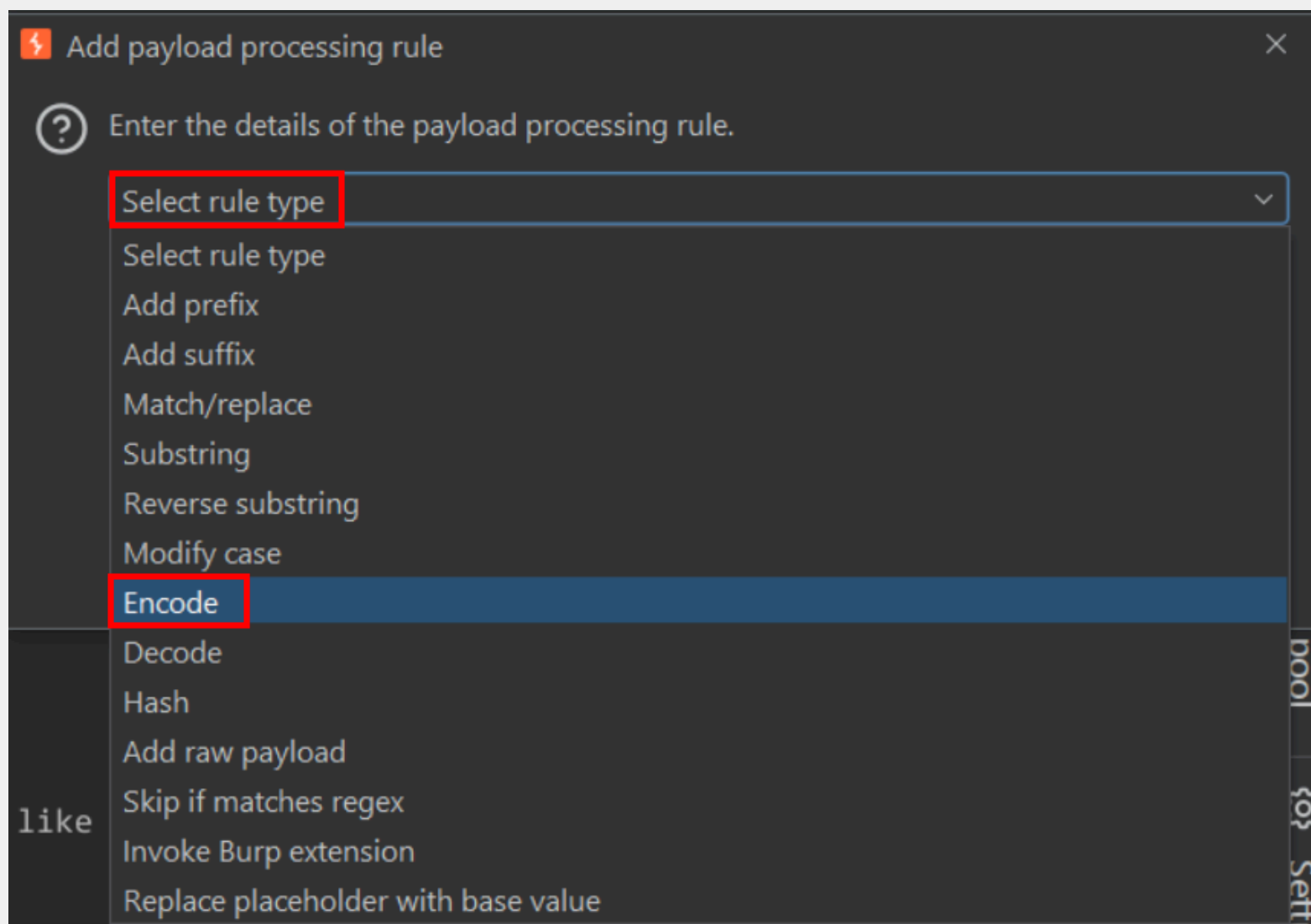
Settings

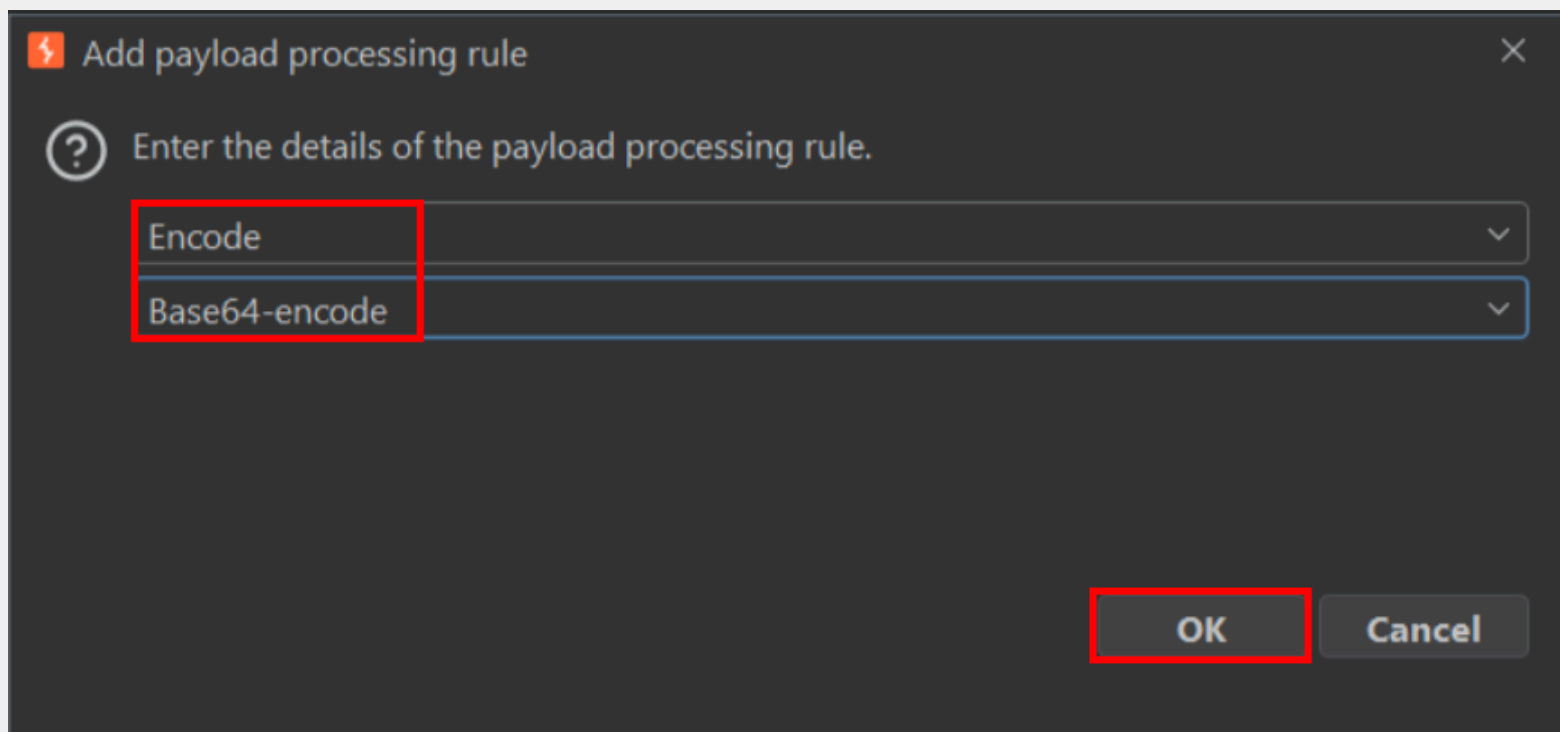
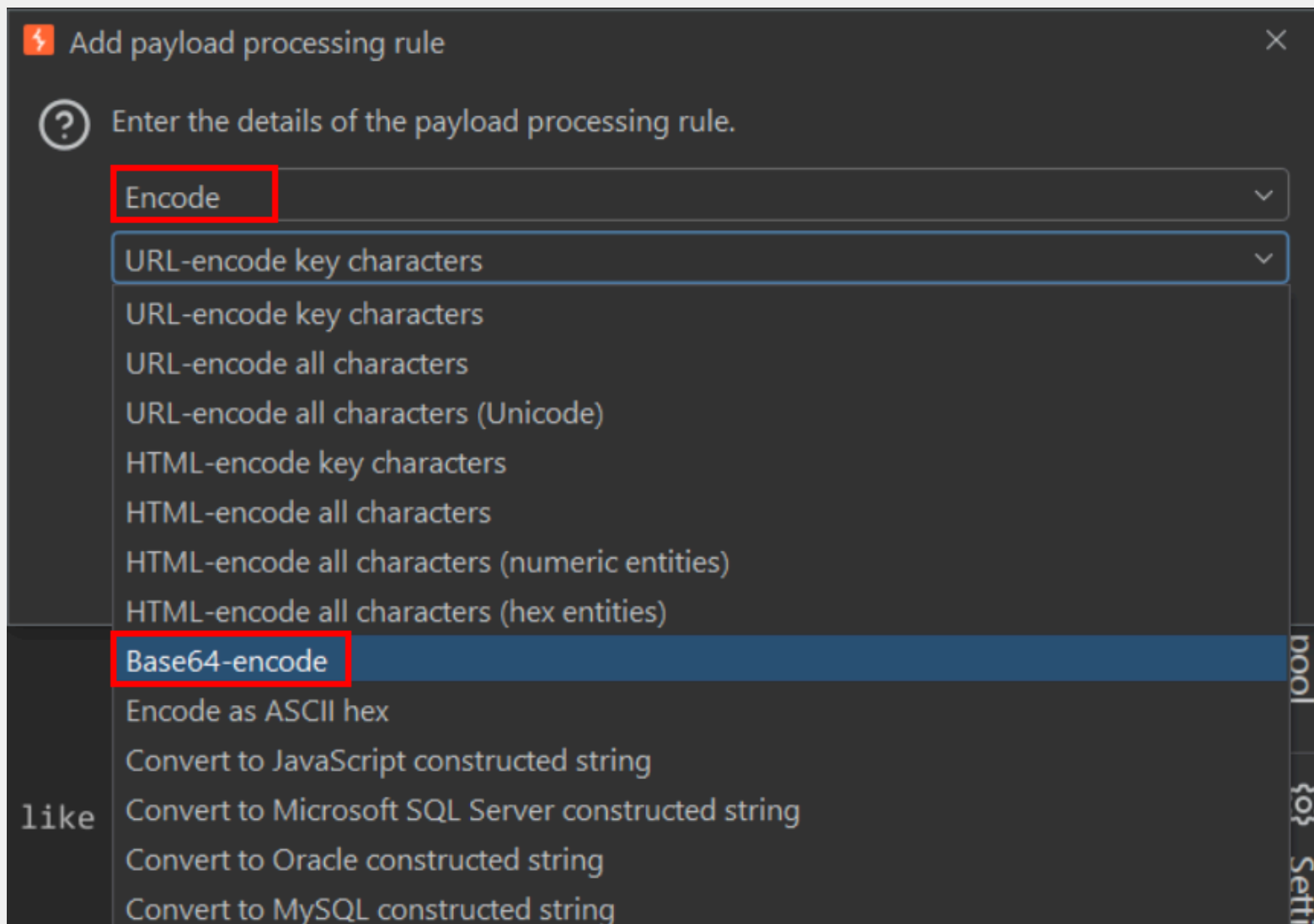


We know then that we need to add a prefix and this is going to be the username. Select rule type then click on Add prefix. The prefix in this case is the username followed by a colon. Enter username followed by a colon (**wiener:**). Hit OK



Then we know that entire string, so that's the username followed by the MD5 hash of the password is going to be base 64 encoded. Let's add another rule. Select rule type Encode and in the lower option select Base64 encode and hit OK.





You can see there are three steps there.

Payloads

Remove

Clear

Deduplicate

Add

Add from list... [Pro version only]

Resource pool

Settings

Payload processing

You can define rules to perform various processing tasks on each payload before it is used.

Add

Edit

Remove

Up

Down

☒	Enabled	Rule
☒		Hash: MD5
☒		Add Prefix: wiener:
☒		Base64-encode

Payload encoding

Let's start attack.

Attack Save 2. Intruder attack of https://0a2700f803bdf8868181b12700ca003b.web-security-academy.net

2. Intruder attack of https://0a2700f803bdf8868181b12700ca003b.web-security-academy.net

Results Positions

Capture filter: Capturing all items Apply capture filter

View filter: Showing all items

Request ^	Payload	Status code	Response receiv...	Error	Timeout	Length	Comment
0		200	179			3363	
1	d2llbmVyOjUxZGMzMGRkYzQ3M2Q0M2E2...	200	177			3363	

Request Response

Pretty Raw Hex

```
1 GET /my-account?id=wiener HTTP/2
2 Host: 0a2700f803bdf8868181b12700ca003b.web-security-academy.net
3 Cookie: stay-logged-in=d2llbmVyOjUxZGMzMGRkYzQ3M2Q0M2E2MDExZTllYmJhNmNhNzcw; session=JjPgFMoPewZTe67NoeHkCo4c10fXwZnL
4 Cache-Control: max-age=0
5 Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "Windows"
8 Accept-Language: en-US,en;q=0.9
9 Upgrade-Insecure-Requests: 1
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
11 Accept:
```

Inspector

Selection 52 (0x34)

Selected text

```
d2llbmVyOjUxZGMzMGRkYzQ3M2Q0M2E2MDExZTllYmJhNmNhNzcw
```

Decoded from: Base64

```
wiener:51dc30ddc473d43a6011e9ebb a6ca770
```

Request attributes 2

Request query parameters 1

Finished

So we can see request 1. We have our payload, which is that base 64 encoded string.

Attack Save 2. Intruder attack of https://0a2700f803bdf8868181b12700ca003b.web-security-academy.net

2. Intruder attack of https://0a2700f803bdf8868181b12700ca003b.web-security-academy.net

Results Positions

Capture filter: Capturing all items Apply capture filter

View filter: Showing all items

Request ^	Payload	Status code	Response receiv...	Error	Timeout	Length	Comment
0		200	179			3363	
1	d2llbmVyOjUxZGMzMGRkYzQ3M2Q0M2E2...	200	177			3363	

Request Response

Pretty Raw Hex Render

Home | My account | Log out

My Account

Your username is **wiener**

Email

Update email

Inspector

Selection 52 (0x34)

Selected text

```
d2llbmVyOjUxZGMzMGRkYzQ3M2Q0M2E2MDExZTllYmJhNmNhNzcw
```

Decoded from: Base64

```
wiener:51dc30ddc473d43a6011e9ebb a6ca770
```

Request attributes 2

Request query parameters 1

Finished

We have a 200 status code. And if we take a look at the response, we'll see the string Update email. Now we didn't need to grep for that because we have a single payload.

However, when we're brute forcing the victim's cookie, it's going to make sense to grep for that string update email so that we can understand which payload was successful in brute forcing the victim's cookie.

This proves that we can create our own stay-logged-in cookie. Also the session cookie should also be verified on the server which is not in this case and we are able to access the account only using stay-logged-in cookie value.

Step 6: Prepare the attack against Carlos

Now we attack Carlos.

There are two things we need to change.

Change 1: Change the account ID

Change:

GET /my-account?id=wiener

to:

GET /my-account?id=carlos

The screenshot shows the Burp Suite Intruder tool interface. The 'Intruder' tab is selected. The 'Sniper attack' dropdown is set to 'Sniper attack'. The target URL is 'https://0a2700f803bdf8868181b12700ca003b.web-security-academy.net'. The 'Update Host header to match target' checkbox is checked. The 'Positions' section shows 'Add \$', 'Clear \$', and 'Auto \$' buttons. The request list shows 16 lines of an HTTP GET request. The request body is highlighted in yellow. The status bar at the bottom shows '1 highlight', '1 payload position', and 'Length: 872'.

Dashboard Target Proxy **Intruder** Repeater Collaborator Sequencer Decoder Comparer Logger Orga

1 2 x +

? Sniper attack Start attack

Target https://0a2700f803bdf8868181b12700ca003b.web-security-academy.net Update Host header to match target

Positions Add \$ Clear \$ Auto \$

```
1 GET /my-account?id=carlos HTTP/2
2 Host: 0a2700f803bdf8868181b12700ca003b.web-security-academy.net
3 Cookie: stay-logged-in=$$; session=JjPgFMoPewZTe67NoeHkCo4c10fXwZnL
4 Cache-Control: max-age=0
5 Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "Windows"
8 Accept-Language: en-US,en;q=0.9
9 Upgrade-Insecure-Requests: 1
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
    Gecko) Chrome/151.0.0.0 Safari/537.36
11 Accept:
    text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apn
    g,/*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-Mode: navigate
14 Sec-Fetch-User: ?1
15 Sec-Fetch-Dest: document
16 Referer: https://0a2700f803bdf8868181b12700ca003b.web-security-academy.net/login
```

? ⚙️ ⬅️ ➡️ Search 1 highlight 1 payload position Length: 872

We are now requesting Carlos's account.

Change 2: Change the cookie prefix

The cookie must also be generated for Carlos.

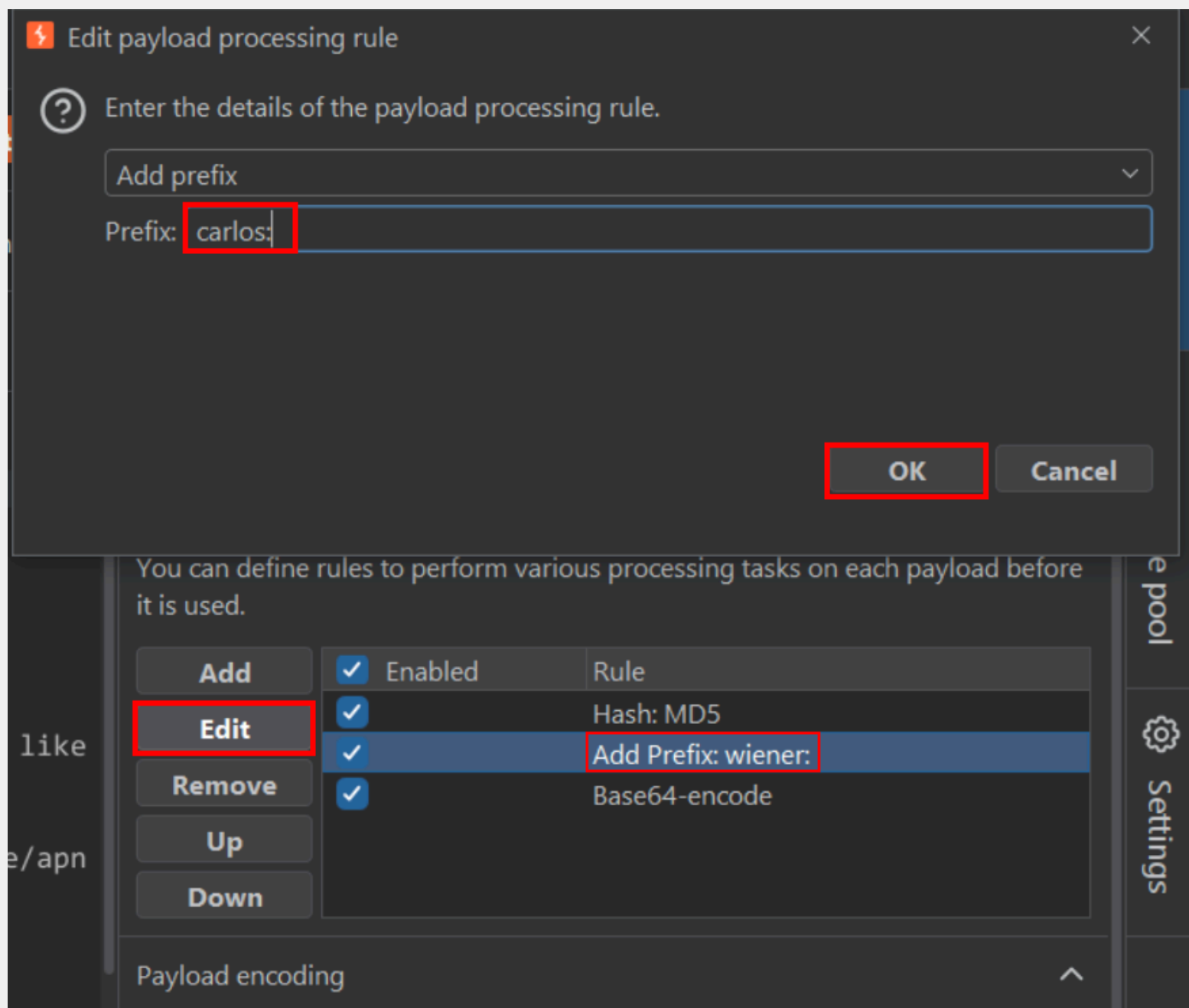
Change the prefix from:

wiener:

to:

carlos:

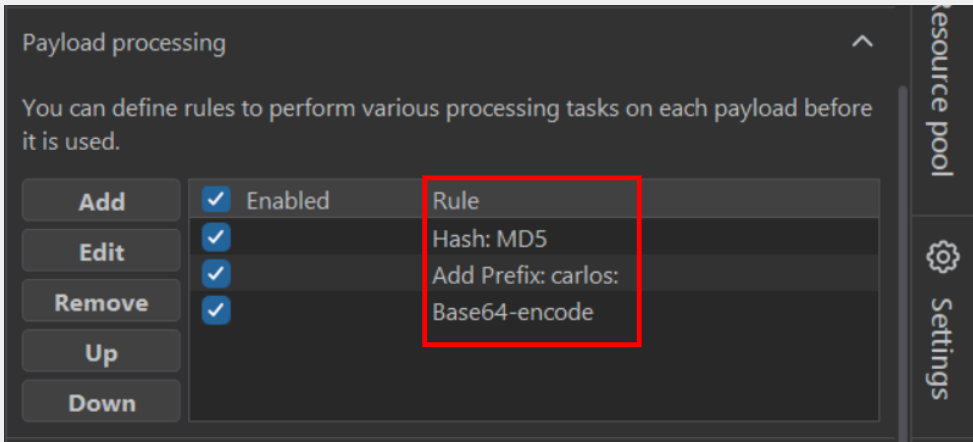
Click on the Add Prefix rule and then Hit Edit and in the prefix value change it from **wiener:** to **carlos:** and Select OK.



The cookie will now be generated as:

```
Base64(carlos:MD5(password))
```

And the rules now look like:



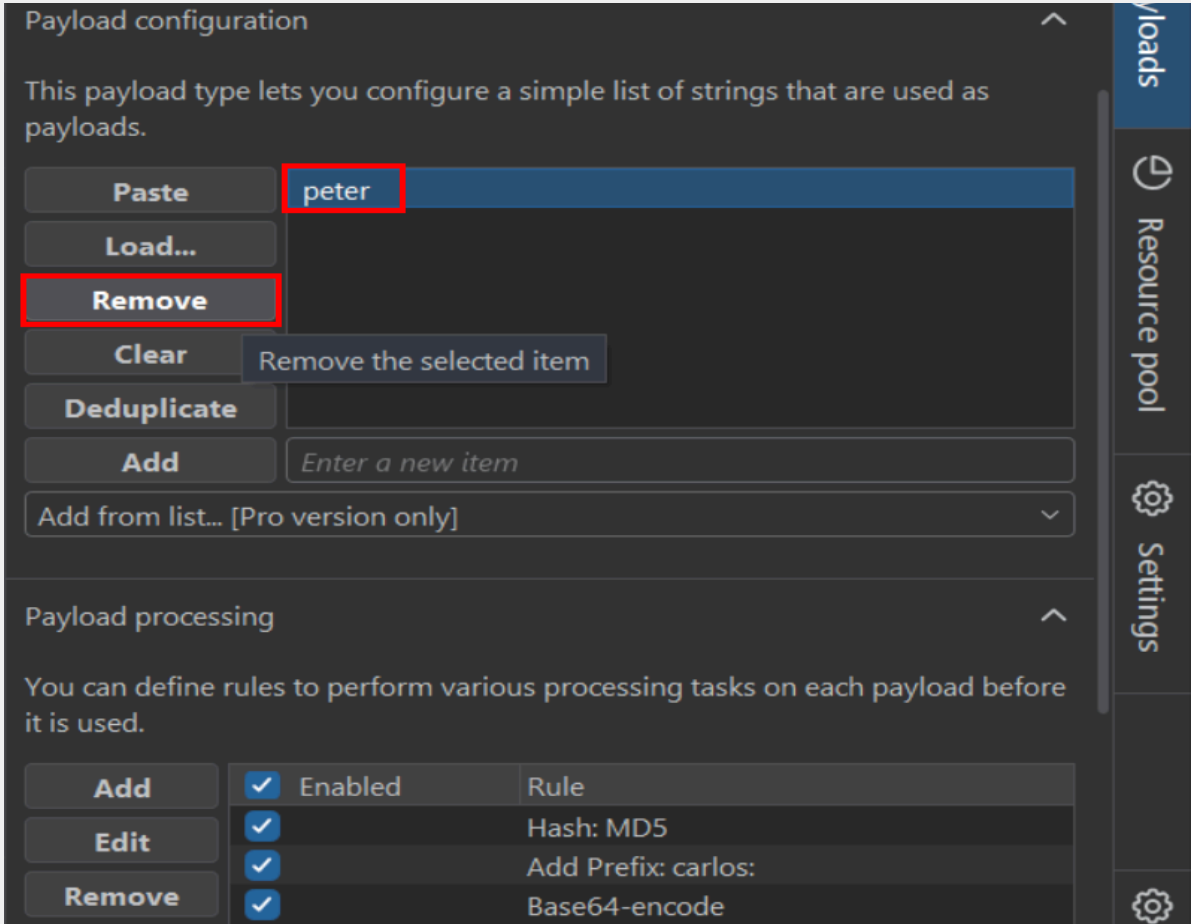
Step 7: Load the candidate passwords

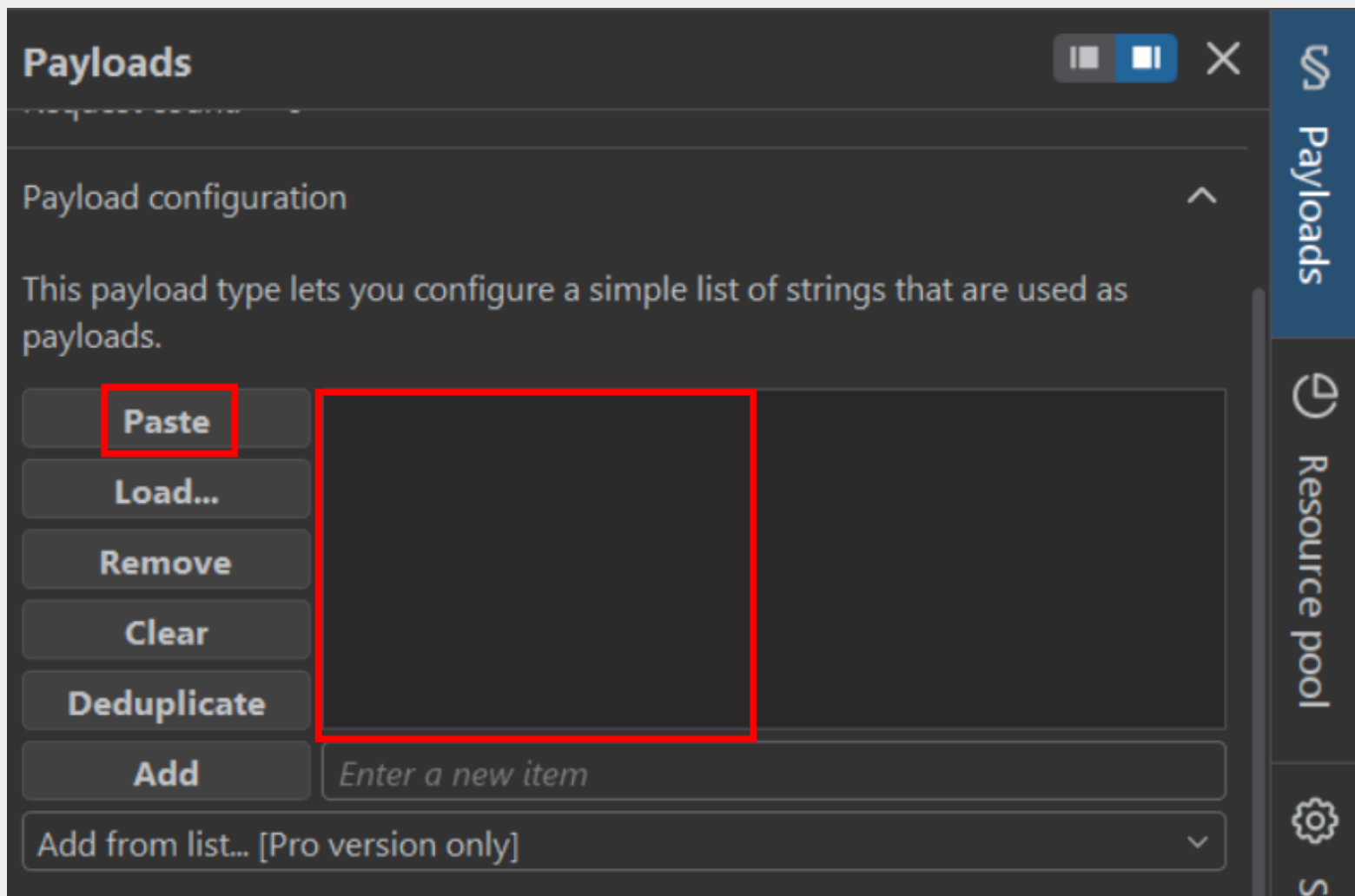
Remove:

```
peter
```

from the payload list.

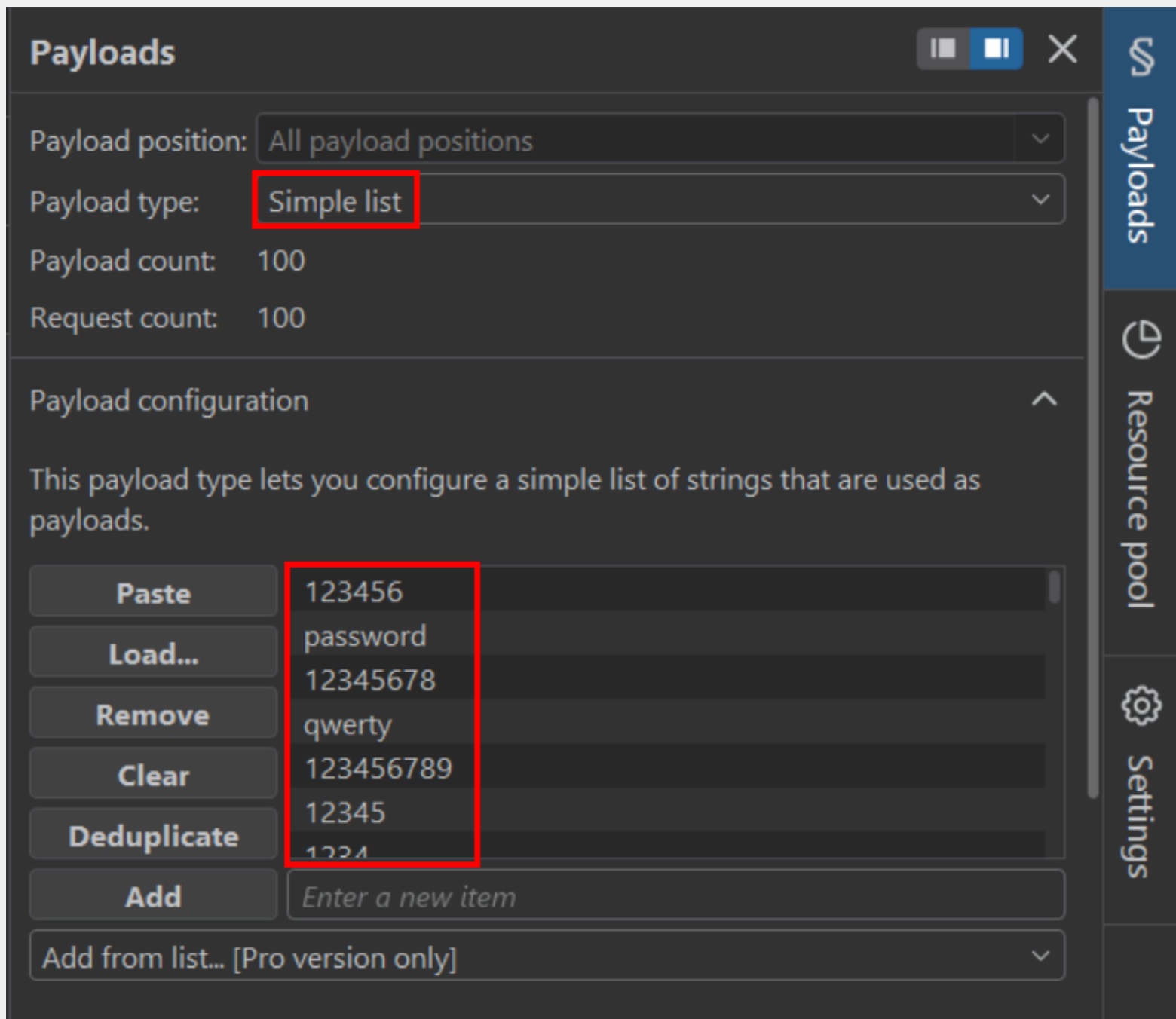
Just select `peter` and click on `Remove`.





Now use PortSwigger's provided **candidate password list**.

Copy the passwords in the candidate password list and click **Paste**.



For every password in the list, Burp will perform:

Password

↓ MD5

MD5(password)

↓ Add carlos:

carlos:MD5(password)

↓ Base64

Base64(carlos:MD5(password))

Each generated value will be sent as Carlos's stay-logged-in cookie.

Step 8: Identify the successful request

Most of the generated cookies will be invalid.

An unsuccessful request will normally redirect us to `/login` because the application does not consider us authenticated.

We need an easy way to find the successful request.

Remember that an authenticated My Account page contains `Update email`

Therefore, configure a **Grep Match** / response search for `Update email`

Go to Settings and then in the Grep - Match section, clear the current payloads by clicking clear and then Yes on the pop-up box.

The screenshot shows the 'Settings' window of a web application. The window has a dark theme and a sidebar on the right with tabs: 'Payloads', 'Resource pool', and 'Settings'. The 'Settings' tab is highlighted with a red box. The main content area is titled 'Settings' and contains the 'Grep - Match' section, which is also highlighted with a red box. The 'Grep - Match' section includes a help icon, a description, a checkbox for 'Flag responses matching these expressions:', a list of expressions, and buttons for 'Paste', 'Load...', 'Remove', 'Clear', and 'Add'. The 'Clear' button is highlighted with a red box, and a tooltip indicates 'Remove all items from the list'. The list of expressions includes 'unknown', 'uid=', 'c:\', 'varchar', 'ODBC', and 'SQL'. Below the list, there is a 'Match type' section with radio buttons for 'Simple string' (selected) and 'Regex'. There are also checkboxes for 'Case sensitive match' and 'Exclude HTTP headers' (checked). The 'Grep - Extract' section is partially visible at the bottom.

Settings

☐ Case-sensitive match

Grep - Match

These settings can be used to flag result items containing specified expressions.

☐ Flag responses matching these expressions:

Paste unknown
Load... uid=
Remove c:\n
Clear varchar
ODBC
SQL

Add Enter a new item

Match type: ☒ Simple string
☐ Regex

☐ Case sensitive match
☒ Exclude HTTP headers

Grep - Extract

Settings



Clear list

Are you sure you want to clear the list?

No

Yes

In the box to the front of the Add button, type in **Update email** because if the response contains that, it means we got logged in and then click Add. Grep - Match works in a way that it find this specific string in all the responses.

Settings

☐ Case-sensitive match



Grep - Match



These settings can be used to flag result items containing specified expressions.

☐ Flag responses matching these expressions:

Paste

Load...

Remove

Clear

Add

Update email

Add the specified item

Match type: ☒ Simple string

☐ Regex

☐ Case sensitive match

☒ Exclude HTTP headers



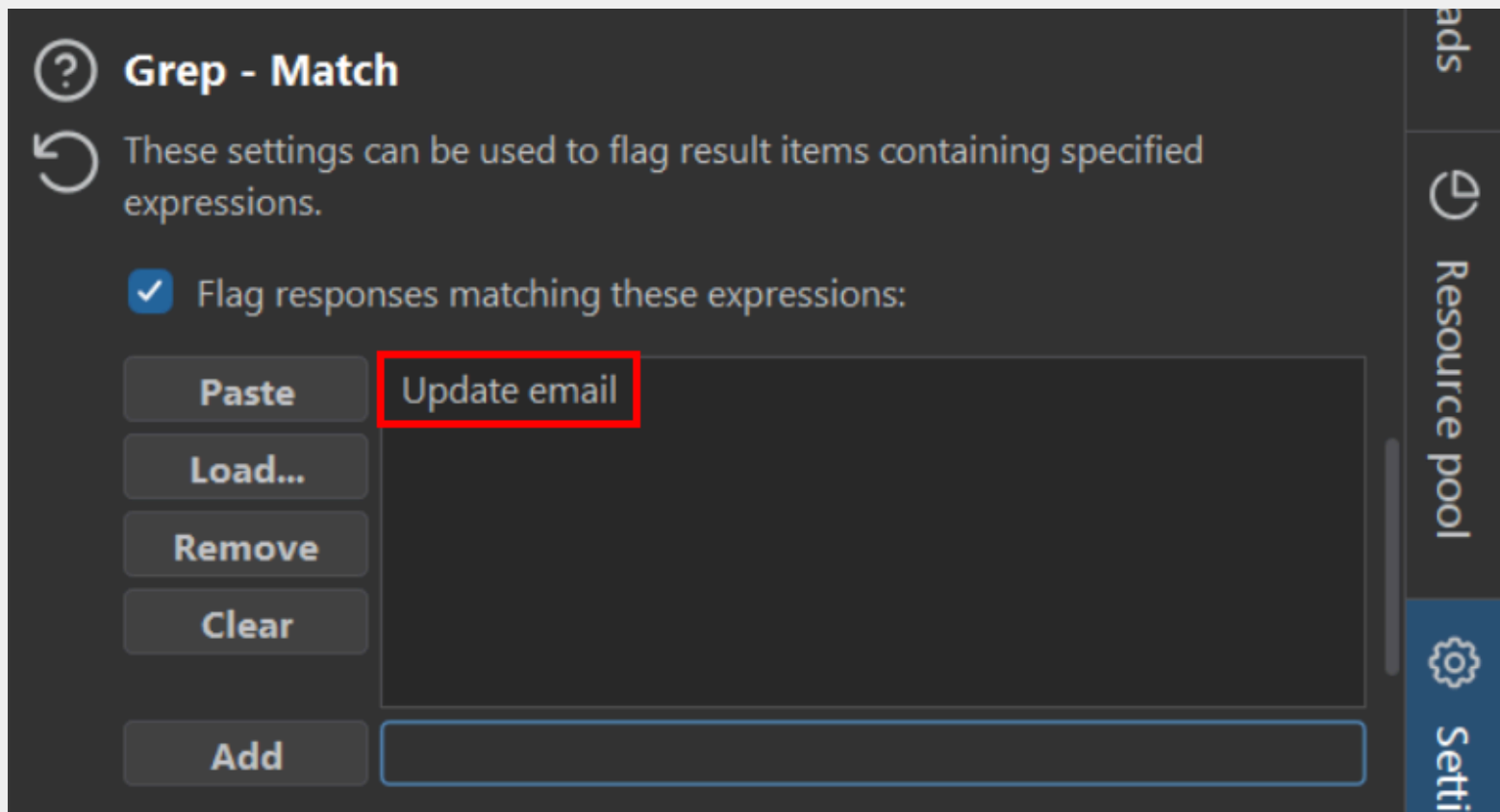
Payloads



Resource pool



Settings



Start the attack.

Step 9: Find the valid cookie

After the attack finishes, look through the results.

Most requests will have a response that redirects to [/login](#).

One request should be different.

It should:

- return a **200** response
- contain **Update email** in the response
- indicate that we successfully authenticated as Carlos

The screenshot shows the Burp Suite interface with the title "9. Intruder attack of https://0a2700f803bdf8868181b12700ca003b.web-security-academy.net". The "Results" tab is active, displaying a list of requests. The first request (index 40) has a status code of 200 and is highlighted. The "Request" tab is selected, showing the raw HTTP request. The "Cookie" header contains the value `stay-logged-in=Y2FybG9zOmQ5YjIzZWJiZjliNDMxZDAwOWEyMGRmNTJlNTE1ZGI1; session=JjPgFmPewZTe67NoeHkCo4c10fXwZnL`. The "Inspector" panel on the right shows the selected text `Y2FybG9zOmQ5YjIzZWJiZjliNDMxZDAwOWEyMGRmNTJlNTE1ZGI1` decoded from Base64 to `carlos:d9b23ebbf9b431d009a20df52e515db5`.

Request	Payload	Status code	Response received	Error	Timeout	Length	Update email	Comment
40	Y2FybG9zOmQ5YjIzZWJiZjliNDMxZDAw...	200	195			3450	1	
0		302	169			173		
1	Y2FybG9zOmUxMGFkYzM5NDliYTU5YWJL...	302	168			173		
2	Y2FybG9zOjVmNGRjYzNiNWFnZyZDY...	302	172			173		
3	Y2FybG9zOjI1ZDU1YWQyODNhYTQwMG...	302	172			173		
4	Y2FybG9zOmQ4NTc4ZWVmODQ1OGNI...	302	171			173		

```
1 GET /my-account?id=carlos HTTP/2
2 Host: 0a2700f803bdf8868181b12700ca003b.web-security-academy.net
3 Cookie: stay-logged-in=Y2FybG9zOmQ5YjIzZWJiZjliNDMxZDAwOWEyMGRmNTJlNTE1ZGI1; session=JjPgFmPewZTe67NoeHkCo4c10fXwZnL
4 Cache-Control: max-age=0
5 Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "Windows"
8 Accept-Language: en-US,en;q=0.9
9 Upgrade-Insecure-Requests: 1
```

Inspector Selection: 52 (0x34)

Selected text: Y2FybG9zOmQ5YjIzZWJiZjliNDMxZDAwOWEyMGRmNTJlNTE1ZGI1

Decoded from: Base64

carlos:d9b23ebbf9b431d009a20df52e515db5

This is the successful password candidate.

The corresponding generated `stay-logged-in` value is Carlos's valid cookie.

This is a close-up of the Inspector panel from the previous screenshot. It shows the "Selected text" as `Y2FybG9zOmQ5YjIzZWJiZjliNDMxZDAwOWEyMGRmNTJlNTE1ZGI1`. Below it, the "Decoded from: Base64" section shows the decoded value `carlos:d9b23ebbf9b431d009a20df52e515db5`, which is highlighted with a red box.

If you want you can also find the password from this hashed string by just copying it and searching it on the Crackstation website but we don't need to do that. We can even access the account without that but that is also a way to find carlos' actual password.

Step 10: Access Carlos's account

Right-click the successful request.

Choose:

Open response in browser

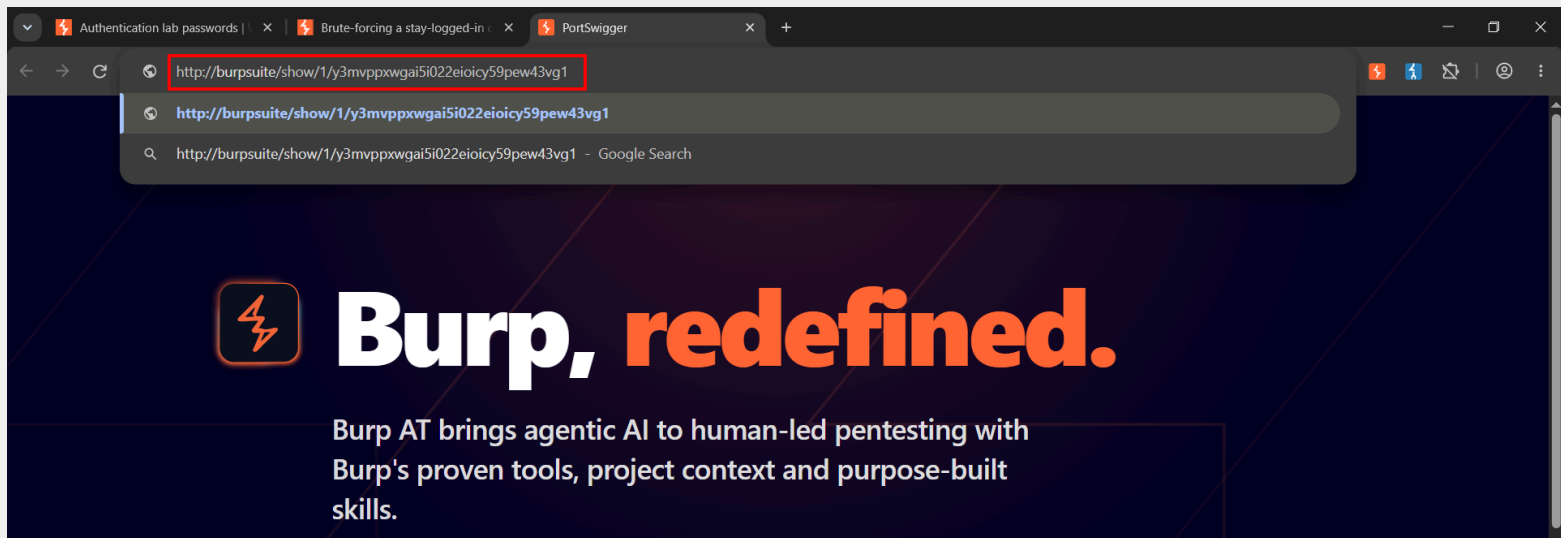
The screenshot shows the Burp Suite interface. At the top, the title bar reads "9. Intruder attack of https://0a2700f803bdf8868181b12700ca003b.web-security-academy.net". Below the title bar, there are tabs for "Results" and "Positions". The "Results" tab is active, showing a list of requests. The first request (index 40) has a status code of 200, which is highlighted with a red box. A right-click context menu is open over this request, with the option "Open response in browser" highlighted. The "Request" tab is selected in the bottom left, showing the raw request details. The "Inspector" panel on the right shows the selected text of the request body, which is a Base64 encoded string. The "Decoded from" dropdown is set to "Base64", and the decoded text is visible below it.

Request	Payload	Status code	Response received
40	Y2FybG9zOmQ5YjZlZWJiNDMxZDAw...	200	195
0		302	169
1	Y2FybG9zOmUxMGFkYzY5NDliYTU5YWJ...	302	168
2	Y2FybG9zOjVmNGRjYzNiNWZhNzY1ZDY...	302	172
3	Y2FybG9zOjI1ZDU1YWQyODNhYTQwMG...	302	172
4	Y2FybG9zOmQ4NTc4ZWVmODQ1OGNl...	302	171

```
1 GET /my-account?id=carlos HTTP/2
2 Host: 0a2700f803bdf8868181b12700ca003b.web-security-academy.net
3 Cookie: stay-logged-in=Y2FybG9zOmQ5YjZlZWJiNDMxZDAwOWEYMGRRNTJlJjJpGFMoPewZTe67NoeHkCo4c10fXwZnL
4 Cache-Control: max-age=0
5 Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "Windows"
8 Accept-Language: en-US,en;q=0.9
9 Upgrade-Insecure-Requests: 1
```

Copy the generated URL and open it in the browser.

The screenshot shows a dialog box titled "Show response in browser". It contains the text: "To show this response in your browser, copy the URL below and paste into a browser that is configured to use Burp as its proxy." Below this text, there is a text input field containing the URL "http://burpsuite/show/1/y3mvppxwgai5i022eioicy59pew43vg1", which is highlighted with a red box. To the right of the input field is a "Copy" button, also highlighted with a red box. Below the input field, there is a checkbox labeled "In future, just copy the URL and don't show this dialog". At the bottom right, there is a "Close" button.



Congratulations, you solved the lab!

My Account

Your username is: carlos

Email

Update email

You should now be accessing: Carlos's My Account

The lab should then be solved.

<https://www.linkedin.com/in/haroonshahid76/>

<https://github.com/haroon505>

=====